Practical 13: PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload

Problem Definition Create simple PHP JSON API endpoints for events or students and

consume them using Fetch API. Perform list, add, update, and delete

operations asynchronously without full page reload.

DB.PHP:-

```
<?php
$conn = new mysqli("localhost", "root", "", "event_db");

if ($conn->connect_error) {
   die("Database connection failed: " . $conn->connect_error);
}

$conn->set_charset("utf8mb4");
?>
```

APAX.HTML:-

```
<!DOCTYPE html>
<html>
<head>
   <title>Event CRUD using Fetch API</title>
</head>
<body>

<h2>Event Management - Fetch API</h2>

<p id="message"></p>

<h3>Add / Update Event</h3>

<form id="eventForm">
   <input type="hidden" id="id">

   Event Name:
   <input type="text" id="name" required>
   <br><br>
```

```
    Event Date:
    <input type="date" id="date" required>
    <br><br>

    Description:
    <textarea id="description" required></textarea>
    <br><br>

    <button type="submit">Save Event</button>
    <button type="button"
onclick="clearForm()">Clear</button>
</form>

<hr>

<h3>All Events</h3>

<table border="1" cellpadding="10">
    <thead>
        <tr>
```

```
            <th>ID</th>
            <th>Name</th>
            <th>Date</th>
            <th>Description</th>
            <th>Actions</th>
        </tr>
    </thead>
    <tbody id="eventList"></tbody>
</table>

<script>
const api = "api.php";

async function loadEvents() {
    try {
        const response = await fetch(api);
        const result = await response.json();

        if (!response.ok || !result.success) {
            throw new Error(result.message || "Could not load
events");
```

```
}

const list = document.getElementById("eventList");
list.innerHTML = "";

result.data.forEach(event => {
    const row = document.createElement("tr");

    [
        event.id,
        event.event_name,
        event.event_date,
        event.description
    ].forEach(value => {
        const cell = document.createElement("td");
        cell.textContent = value;
        row.appendChild(cell);
    });

    const actions = document.createElement("td");
```

```
            const edit = document.createElement("button");
            edit.textContent = "Edit";
            edit.onclick = () => editEvent(event);

            const del = document.createElement("button");
            del.textContent = "Delete";
            del.onclick = () => deleteEvent(event.id);

            actions.append(edit, del);
            row.appendChild(actions);
            list.appendChild(row);
        });
    } catch (error) {
        showMessage(error.message);
    }
}

document.getElementById("eventForm").onsubmit = async function(e) {
    e.preventDefault();
```

```
    const id = document.getElementById("id").value;

    const data = {
        event_name: document.getElementById("name").value,
        event_date: document.getElementById("date").value,
        description:
document.getElementById("description").value
    };

    if (id) {
        data.id = Number(id);
    }

    try {
        const response = await fetch(api, {
            method: id ? "PUT" : "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(data)
        });

        const result = await response.json();
```

```
        if (!response.ok || !result.success) {
            throw new Error(result.message || "Operation
failed");
        }

        showMessage(result.message);
        clearForm();
        await loadEvents();
    } catch (error) {
        showMessage(error.message);
    }
};

function editEvent(event) {
    document.getElementById("id").value = event.id;
    document.getElementById("name").value =
event.event_name;
    document.getElementById("date").value =
event.event_date;
    document.getElementById("description").value =
event.description;
```

```
    showMessage("Editing event ID " + event.id);
}

async function deleteEvent(id) {
    if (!confirm("Are you sure you want to delete this event?"))
{
        return;
    }

    try {
        const response = await fetch(api, {
            method: "DELETE",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ id: Number(id) })
        });

        const result = await response.json();

        if (!response.ok || !result.success) {
            throw new Error(result.message || "Delete failed");
```

```
        }

        showMessage(result.message);
        await loadEvents();
    } catch (error) {
        showMessage(error.message);
    }
}

function clearForm() {
    document.getElementById("eventForm").reset();
    document.getElementById("id").value = "";
}

function showMessage(message) {
    document.getElementById("message").textContent =
message;
}

loadEvents();
</script>
```

```
</body>
</html>
```

API.PHP

```
<?php
header("Content-Type: application/json");

include 'db.php';

$method = $_SERVER['REQUEST_METHOD'];

function sendResponse($success, $message, $data = null) {
    echo json_encode([
        "success" => $success,
        "message" => $message,
        "data" => $data
    ]);
    exit;
}
```

```
if ($method == "GET") {

    $result = $conn->query(
        "SELECT id, event_name, event_date, description
         FROM events ORDER BY id DESC"
    );

    $events = [];

    while ($row = $result->fetch_assoc()) {
        $events[] = $row;
    }

    sendResponse(true, "Events fetched successfully",
$events);
}

$input = json_decode(file_get_contents("php://input"), true);

if (!is_array($input)) {
    sendResponse(false, "Invalid JSON data");
```

```
}

if ($method == "POST") {

    $name = trim($input['event_name'] ?? '');
    $date = $input['event_date'] ?? '';
    $description = trim($input['description'] ?? '');

    if ($name == "" || $date == "" || $description == "") {
        sendResponse(false, "All fields are required");
    }

    $stmt = $conn->prepare(
        "INSERT INTO events (event_name, event_date,
description)
         VALUES (?, ?, ?)"
    );
    $stmt->bind_param("sss", $name, $date, $description);

    if ($stmt->execute()) {
        sendResponse(true, "Event added successfully");
```

```
    }

    sendResponse(false, "Failed to add event");
}

if ($method == "PUT") {

    $id = filter_var($input['id'] ?? null, FILTER_VALIDATE_INT);
    $name = trim($input['event_name'] ?? '');
    $date = $input['event_date'] ?? '';
    $description = trim($input['description'] ?? '');

    if (!$id || $id < 1 || $name == "" ||
        $date == "" || $description == "") {
        sendResponse(false, "Valid ID and all fields are
required");
    }

    $stmt = $conn->prepare(
        "UPDATE events
         SET event_name = ?, event_date = ?, description = ?
```

```
        WHERE id = ?"
    );
    $stmt->bind_param("sssi", $name, $date, $description,
$id);

    if ($stmt->execute() && $stmt->affected_rows >= 0) {
        sendResponse(true, "Event updated successfully");
    }

    sendResponse(false, "Failed to update event");
}

if ($method == "DELETE") {

    $id = filter_var($input['id'] ?? null, FILTER_VALIDATE_INT);

    if (!$id || $id < 1) {
        sendResponse(false, "Valid event ID is required");
    }
```

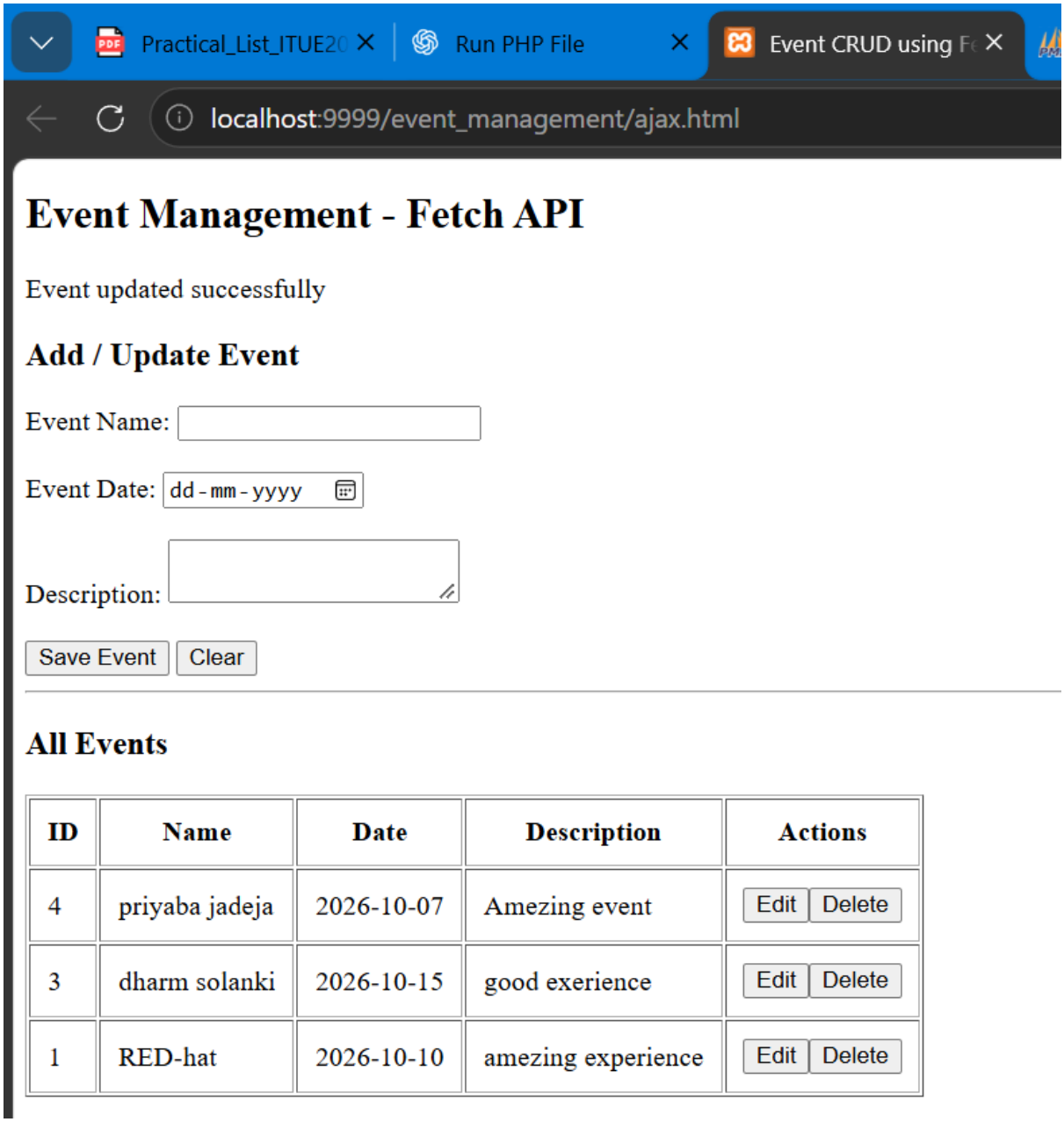
```
    $stmt = $conn->prepare("DELETE FROM events WHERE id =
?");
    $stmt->bind_param("i", $id);
    $stmt->execute();

    if ($stmt->affected_rows > 0) {
        sendResponse(true, "Event deleted successfully");
    }

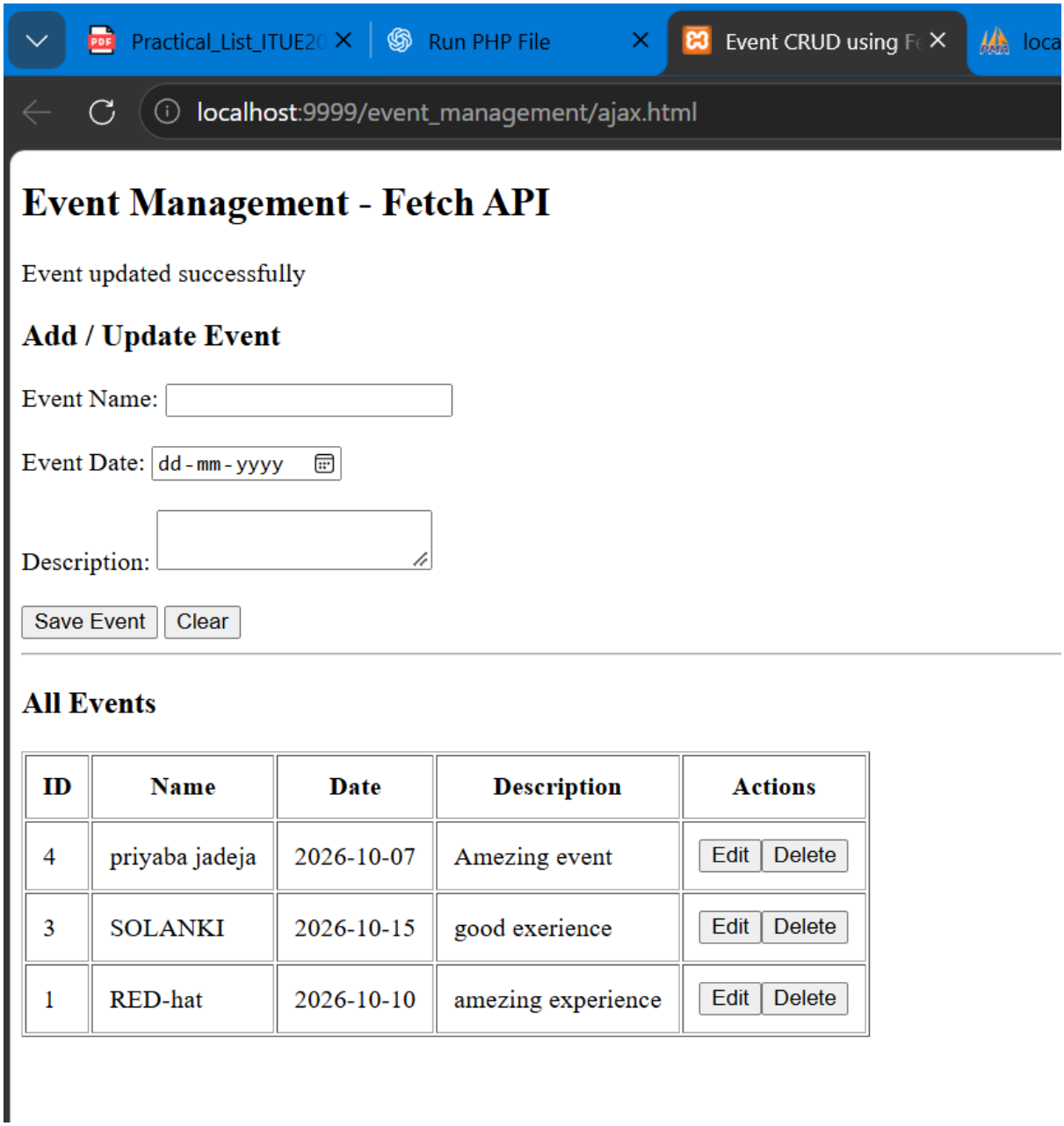
    sendResponse(false, "Event not found");
}

http_response_code(405);
sendResponse(false, "Method not allowed");
?>
```

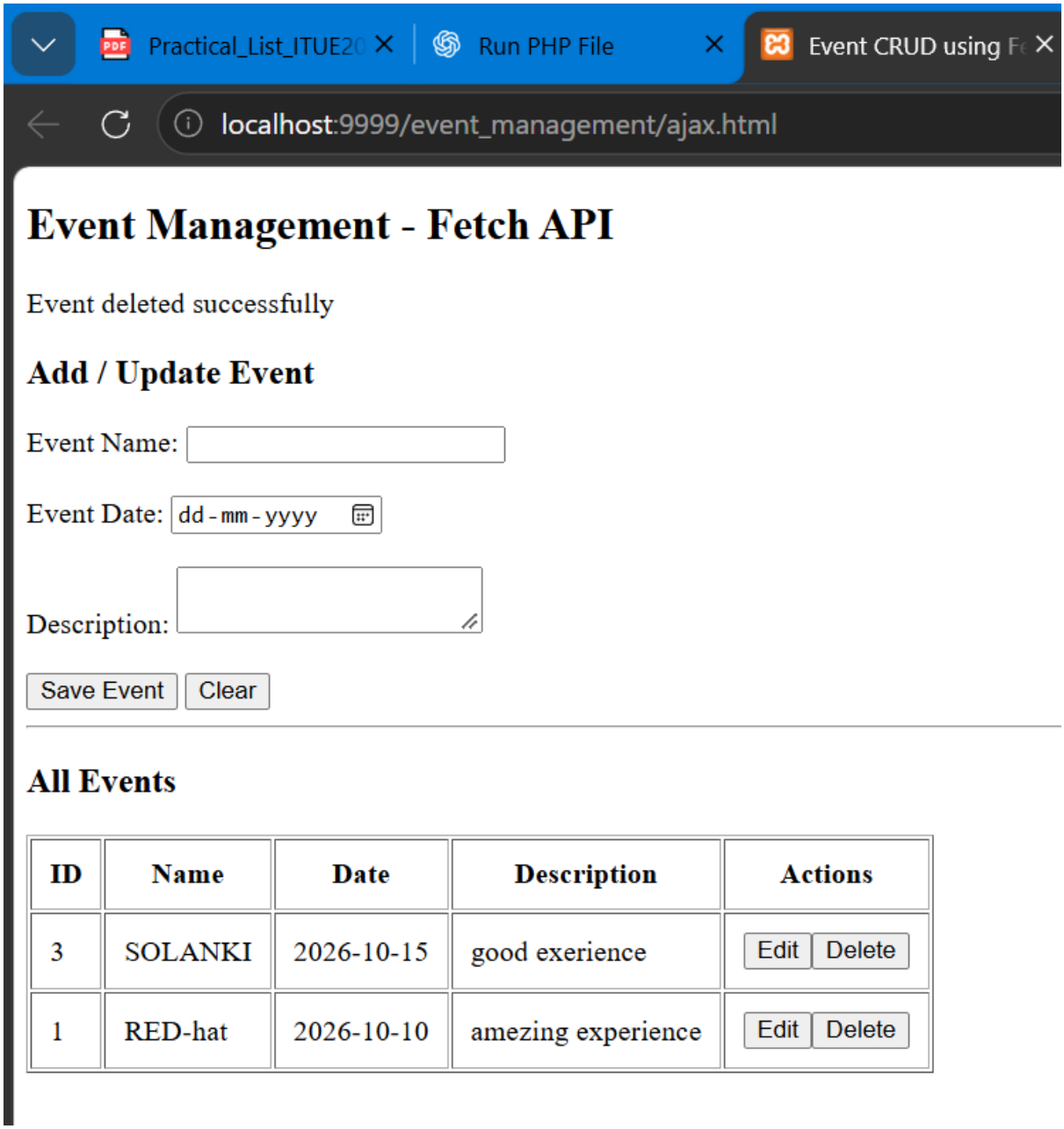
OUTPUT:-

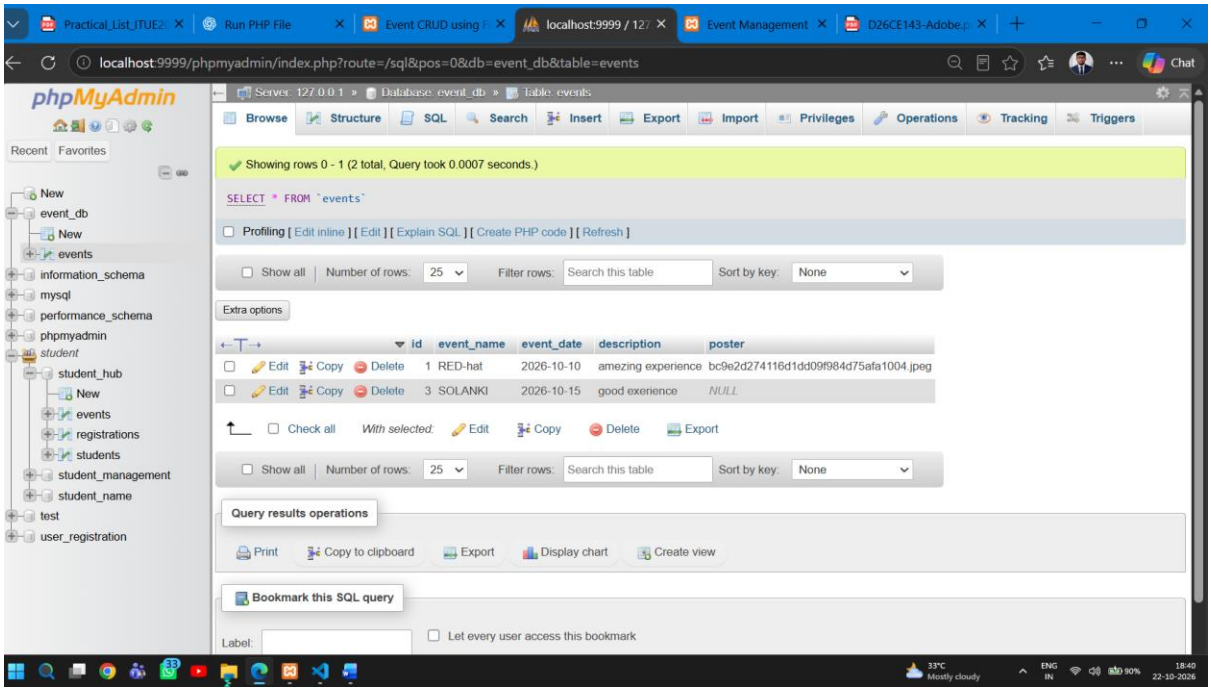


After editing



After delete:-


Practical_List_ITUE2
Run PHP File
Event CRUD using F
localhost:9999/event_management/ajax.html
Event Management - Fetch API
Event deleted successfully
Add / Update Event
Event Name:
Event Date: dd-mm-yyyy
Description:
Save Event
Clear
All Events
ID
Name
Date
Description
Actions
3
SOLANKI
2026-10-15
good exerience
Edit
Delete
1
RED-hat
2026-10-10
amezing experience
Edit
Delete



1. Do PHP endpoints return valid JSON responses?

Ans: Yes, PHP endpoints return valid JSON responses that the frontend can understand.

2. Are GET and POST requests handled properly?

Ans: Yes, GET is used to fetch records, and POST is used to add or update records.

3. Are loading, success, and error states shown on the frontend?

Ans: Yes, the frontend displays loading, success, and error messages to the user.

4. Is input validation performed before database operations?

Ans: Yes, input validation checks that the entered data is correct before performing database operations.

Tools/Technology to be Used :-

VSCODE , PHP, MySQL, JavaScript Fetch API, browser network tab